RLHF & Logic Verification Framework

Project plan, software architecture, and execution visuals

Prepared from your uploaded project brief.

Recommended build sequence: verifier-centric MVP first, then scale to richer feedback and formal verification.

---

1. Executive recommendation

Build this as a verifier-centric reasoning system, not just another fine-tuned chatbot. The project becomes much stronger when the model emits structured traces, a verifier checks those traces, and the verification signals feed back into training and selection.

Recommended MVP: arithmetic plus symbolic logic. That gives you fast deterministic checks first, meaningful failure diagnostics second, and a clean path to Z3 before you touch Lean.

2. Project thesis

- Reasoning should be represented as structured JSON steps with dependencies.
- Correctness should be checked by explicit validators rather than trusted implicitly.
- Verifier outputs should become first-class training signals.
- The demo should expose both the answer and the verification report.

3. Recommended build phases

Phase	Goal	Primary outputs	Why it matters
Phase 1	Foundation + baseline SFT	Repo scaffold, schema, baseline model, Tier A verifier	Gets you to a working end-to-end loop quickly
Phase 2	Verifier-guided optimization	Preference pairs, DPO training, best-of-N selection	Shows RLHF-style alignment instead of plain prompting
Phase 3	Formal logic expansion	Z3 integration, counterexample diagnostics	Makes the project technically distinctive
Phase 4	Research polish	Evaluation harness, ablations, demo UI, writeup	Turns the build into a portfolio artifact

4. Architecture overview

The architecture should isolate inference, verification, storage, training, evaluation, and presentation so each layer can be tested and iterated independently.

Figure 1. System architecture

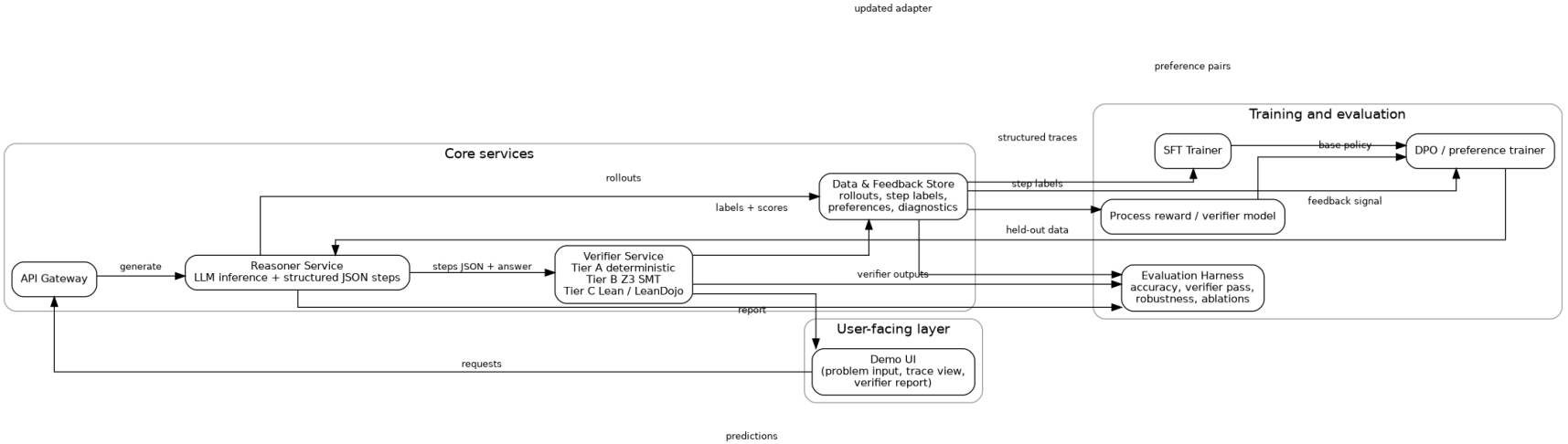
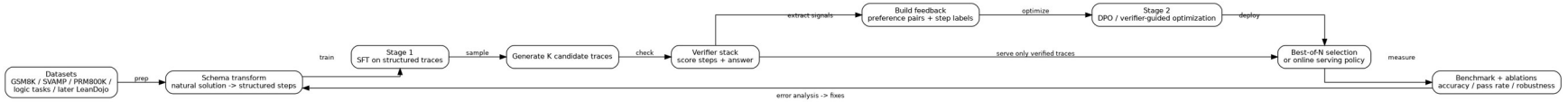


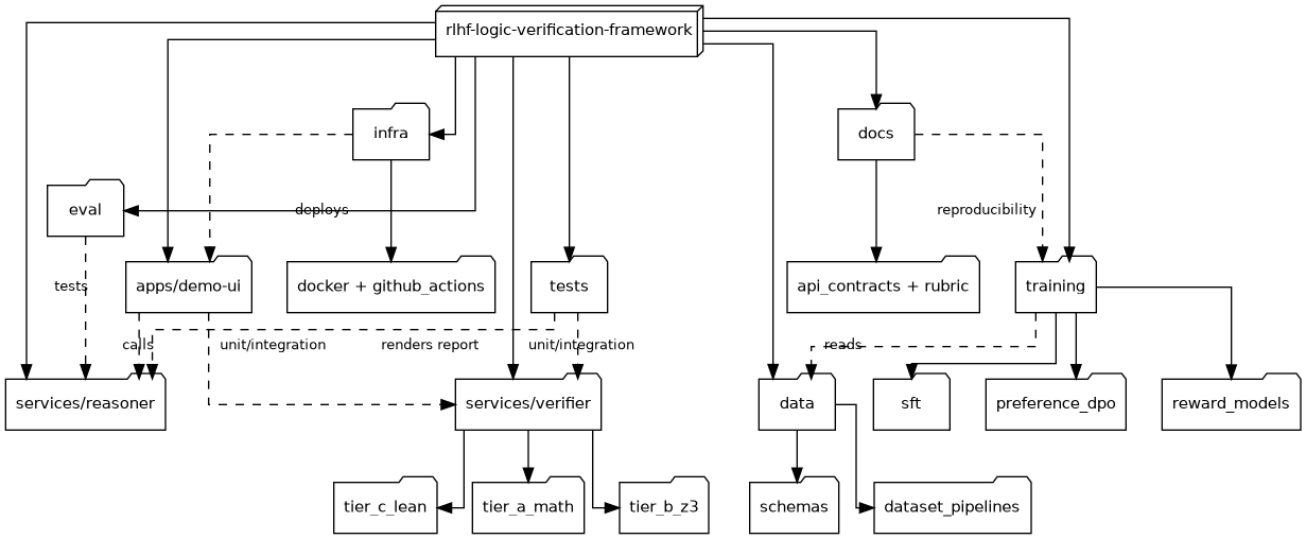
Figure 2. Training and verification feedback loop



5. Software layout

Use a repository layout that reads like a real applied ML system rather than a notebook pile: apps, services, training, evaluation, data pipelines, infrastructure, and docs.

Figure 3. Repository and module structure



6. Execution roadmap

This schedule assumes focused part-time or full-time execution over roughly twelve weeks. Lean is a stretch goal and should not block the main loop.

Figure 4. Suggested 12-week execution plan

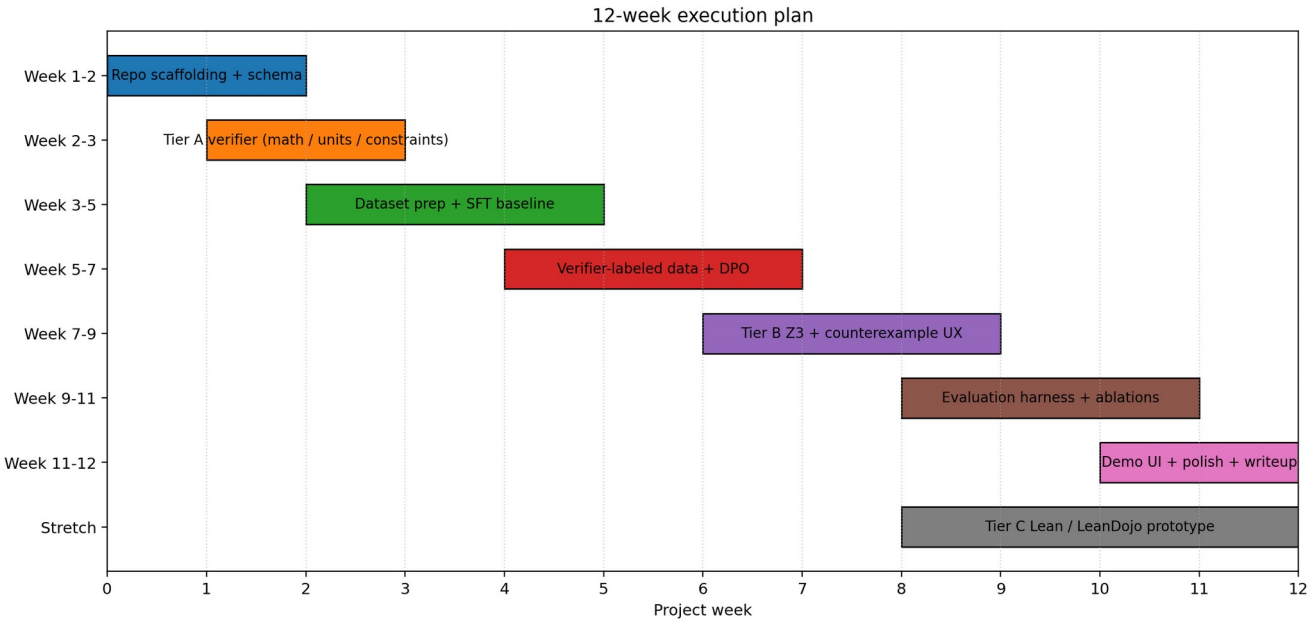
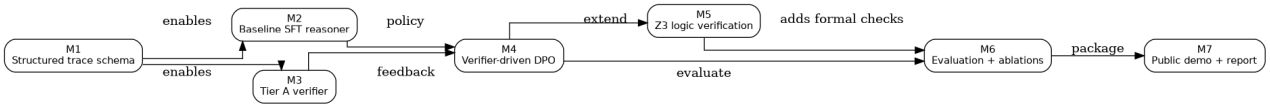


Figure 5. Milestone dependency graph



7. Technical stack

Area	Recommendation
Base model	Start with a small open model in the 2B to 8B range
Fine-tuning	LoRA or QLoRA
Training workflow	SFT first, then DPO
Verification Tier A	Deterministic arithmetic and constraint checks
Verification Tier B	Z3 SMT
Verification Tier C	Lean / LeanDojo later, only after the MVP works
Serving	FastAPI backend plus a simple web UI
Infrastructure	Docker, tests, GitHub Actions

8. Metrics to report

- Final-answer accuracy on held-out tasks
- Verifier pass rate
- Step-level precision and recall when labels are available
- Best-of-N improvement over single-sample inference
- Ablation results across SFT-only, SFT + DPO, and verifier-selection variants
- Failure-case diagnostics with counterexamples or verifier error traces

9. What to show recruiters

- A live demo where a prompt is solved, checked, and diagnosed in one interface
- A clean repository with tests, Docker, CI, and reproducible configs
- A mini research report with the graphs in this document plus ablation tables
- A README that explains why verifier-guided training matters

10. Recommended first milestone

Build the smallest complete loop first: schema -> baseline reasoner -> Tier A verifier -> stored rollouts -> simple evaluation page. Do not start with Lean. Get one narrow domain working beautifully, then expand.